

Program Repair and Automated Debugging with Code LLMs

Abstract

Program repair and automated debugging have evolved from tightly engineered search, template, and synthesis systems into a broader ecosystem in which code-specialized large language models act as patch generators, bug localizers, execution interpreters, critics, and software agents. This shift is substantial, but it is not merely a story of replacing symbolic techniques with neural models. Rather, recent progress shows that the most effective systems combine parametric knowledge from code LLMs with structured feedback from testing, fault localization, repository retrieval, static analysis, execution traces, and interactive tools [1-42]. Classical automated program repair (APR) established the key abstractions of localization, patch generation, validation, patch ranking, and overfitting analysis, while automated debugging contributed spectrum-based localization, delta debugging, execution slicing, and richer failure feedback [43-52]. Learning-based APR then reframed repair as translation, edit prediction, and neural search, creating the immediate methodological bridge to contemporary code LLMs [53-64].

This survey reviews that transition and argues that modern repair systems are best understood as **feedback-driven code reasoning systems** rather than as simple code generators. In this view, repair performance depends less on raw next-token accuracy than on how effectively a system can acquire, represent, and exploit debugging evidence. Prompt-based systems exploit in-context repair knowledge; fine-tuned systems adapt model parameters to patch distributions; procedural systems alternate generation and validation; and agentic systems decide what evidence to collect, which tools to invoke, and when to stop [65-155]. Across these families, the central technical tension is between flexibility and control: general-purpose LLMs offer broad repair priors, but robust debugging still requires explicit constraints, tool use, and evaluation designs that guard against data leakage, over-editing, and test-suite overfitting [20-26, 82-85, 122-128, 141-175].

The survey is organized around a unified perspective on methods, challenges, and prospects. It first revisits foundational APR and automated debugging paradigms, then synthesizes the emerging literature on code LLMs for repair, iterative debugging, repository-level bug fixing, and agentic software engineering. It concludes by identifying the main open problems: semantic correctness beyond weak tests, precise and economical context selection, trustworthy patch assessment, scalable repository-level reasoning, multilingual and low-resource repair, security and privacy, and human-centered integration into real software workflows [35-42, 81-82, 122-128, 141-175].

Introduction

Automated program repair has long pursued a deceptively simple goal: given a faulty program and some notion of desired behavior, generate a modification that fixes the fault. In practice, this objective decomposes into several hard subproblems: locating the defect, constructing a repair space, ranking or searching candidate edits, validating candidates against an oracle, and determining whether a plausible patch is actually correct [1-42]. Early work explored game-theoretic formulations, model repair, search-based mutation, evolutionary synthesis, runtime patching, and specification-guided correction [1-14]. The most influential line in general-purpose APR became test-suite-based repair, exemplified by GenProg

and its descendants, which used failing and passing tests as an implicit specification and searched over edit operators until a candidate passed the available suite [13-16]. Subsequent work diversified into template-based, semantics-based, constraint-based, and redundancy-based approaches, each improving one or more aspects of the repair pipeline while also exposing deeper limitations in fault localization, donor-code retrieval, and semantic correctness [15-35].

A second line of work, often discussed separately, focused on automated debugging rather than direct patch synthesis. Debugging research emphasized failure isolation, spectrum-based fault localization, cause reduction, suspiciousness ranking, and runtime reasoning [43-48]. That distinction matters. Repair is about producing a change; debugging is about producing evidence, hypotheses, and explanations. Classical APR often treated debugging signals as a preprocessing stage, whereas recent code-LLM systems increasingly blur the boundary: they localize, explain, test, simulate, critique, and repair in one loop [49-52]. This convergence is one of the main themes of the modern literature.

Learning-based APR created the immediate precursor to the Code-LLM era. Neural machine translation and encoder-decoder models recast repair as translation from buggy code to fixed code or from faulty context to edit sequence [53-64]. Systems such as SequenceR, CoCoNuT, CURE, and Recoder demonstrated that data-driven models could capture broad repair operators without manually enumerating templates, especially for single-hunk bugs and localized edits [55-58]. Yet these systems still suffered from limited context, syntactic validity problems, sparse supervision on semantic correctness, and weak generalization across projects and bug types [53-64]. Their importance lies in showing that repair could be learned, but also in clarifying why pure end-to-end translation was not enough.

Code LLMs changed the field because they brought three capabilities that earlier repair models largely lacked. First, they offered much stronger prior knowledge about APIs, programming idioms, and latent bug-fix regularities via large-scale pretraining on code and natural language [65-82]. Second, they enabled more general instruction following, making prompt-based repair and interactive debugging feasible without task-specific retraining [72-82]. Third, they made tool-mediated reasoning natural: an LLM can read logs, summarize traces, plan steps, invoke tests, inspect files, and incorporate structured outputs from analyzers or debuggers [49-52, 92-128, 141-155]. As a result, the modern question is no longer whether language models can repair code at all, but rather which combinations of model adaptation, external evidence, and control policies yield reliable and economical repair.

The literature has therefore moved from asking whether LLMs outperform traditional APR on selected benchmarks to examining **how** they repair, **when** they fail, and **what kind of debugging signals** best steer them. Prompting studies showed that zero-shot and few-shot repair can surpass prior systems on some benchmark subsets [83-91]. Objective-alignment and analysis-augmented methods showed that failing tests, compiler feedback, slices, or traces can be more useful than statement-level fault localization assumptions [92-103, 116-124]. Conversational and iterative pipelines then demonstrated that repair quality improves when the model can refine candidate patches under feedback loops [93-95, 113-121, 137-140]. Finally, agentic repository-level systems such as SWE-Agent, Agentless, AutoCodeRover, RepairAgent, OpenHands, MAGIS, SWE-Search, and Learn-by-Interact reframed APR as software-task solving in a partially observable environment [141-155].

This survey uses the term **Code LLMs** broadly to include general large language models specialized for code, code-pretrained encoder-decoder and decoder-only models, and modern post-trained coding agents. It focuses on program repair and automated debugging as intersecting areas, rather than treating repair as a narrow patch-synthesis task. The survey’s core claim is that the field has entered a new phase in which progress depends on closing the loop between generation and evidence. Better patch generators alone are no longer sufficient. The frontier is being shaped by hybrid systems that combine

LLM priors with debugging artifacts, structured search, criticism, and runtime interaction [49-52, 92-128, 141-175].

Methods

Foundational Automated Repair and Debugging Paradigms

The foundational APR literature established four enduring methodological families.

Search-based and generate-and-validate repair. Early systems formulated patch generation as search over mutation operators, genetic transformations, or heuristic edit spaces and used tests as the acceptance criterion [6-16]. These methods were attractive because they required only executable code and a test suite, not formal specifications. GenProg and related systems showed that plausible patches could often be synthesized automatically, initiating the modern APR field [13-16]. Follow-up work refined candidate generation, incorporated code transfer from existing programs, mined repair models from histories, and emphasized simplicity or semantic relevance in patch search [22-35]. This family remains conceptually important because many LLM repair loops still operate as sophisticated generate-and-validate procedures, except that the proposal distribution is now a language model rather than a handcrafted mutation space.

Template-based and redundancy-based repair. Template systems such as PAR, Nopol, TBar, AVATAR, CapGen, ELIXIR, and SimFix exploited recurring fix patterns, contextual donor code, or statistically guided ingredient selection [15,17,19,24,43,62-64]. The central hypothesis was that many real bugs can be fixed by reusing or instantiating common edit schemas. Code LLMs inherit and amplify this idea. Their parametric memory stores enormous latent template libraries, but current LLM methods often still benefit from explicit template guidance, retrieval, or edit control when precision matters [83-103, 116-124]. In that sense, LLM-based APR extends rather than abolishes template-based reasoning.

Semantics-based and synthesis-based repair. Work such as SemFix, DirectFix, Angelix, staged synthesis, and semantic code search aimed to move beyond syntactic plausibility toward repairs constrained by symbolic reasoning, path conditions, or richer semantic objectives [16,22-23,29-31]. These methods were typically more precise than local mutations but more computationally demanding and harder to scale. Their legacy is visible in today’s analysis-augmented and reward-shaped Code-LLM systems, where static or dynamic analysis provides a semantics-sensitive signal to an otherwise neural generator [92, 101, 108-115, 120-128].

Automated debugging and fault localization. Classical debugging contributed techniques for isolating failures, ranking suspicious statements, and extracting minimally relevant evidence [43-48]. Spectrum-based localization, delta debugging, and cause-effect reasoning were designed for diagnosis, but they became indispensable in APR pipelines because repair quality is bottlenecked by localization quality. Much of the modern LLM literature challenges the pipeline assumption that repair should always start from statement-level localization: some works argue for direct whole-function or whole-file debugging; others reintroduce localization through retrieved context, suspiciousness rankings, or interactive tool use [92, 117-124, 141-155]. The deeper lesson persists: effective repair requires evidence concentration. Whether the concentrator is Tarantula, Ochiai-like ranking, a slice, a trace, a knowledge graph, or a repository agent, the objective is to put the right evidence in front of the repair model.

Learning-based APR emerged as a transitional family that drew on all four traditions. Neural machine translation methods learned bug-fix mappings from corpora of commits or curated fix datasets [53-59]. Context-enriched models improved tokenization, search constraints, edit representation, and project-

specific identifiers, leading to stronger performance on standard benchmarks [55-64]. Yet even the best pre-LLM neural repair systems were predominantly local, single-function, and benchmark-centered. Their evaluation often assumed known buggy hunks or heavily localized repair contexts, which limited ecological validity for repository-scale debugging [21,25-26,53-64].

From this foundational literature, five methodological lessons carry directly into the Code-LLM era. First, **repair is inseparable from localization and validation**. Second, **weak test suites produce patch overfitting**. Third, **repair spaces need strong priors or constraints** to avoid combinatorial explosion. Fourth, **context quality matters as much as generation quality**. Fifth, **evaluation must distinguish plausible from semantically correct patches** [20-26,35-42]. Contemporary Code-LLM systems repeatedly rediscover these lessons, often with new names and larger models.

Code LLMs for Repair and Debugging

The Code-LLM literature rests on a rapidly maturing backbone ecosystem. Transformer architectures made large-scale sequence modeling practical [65], and code-specific pretraining then produced models such as CodeBERT, PLBART, GraphCodeBERT, CodeT5, UniXcoder, InCoder, Codex, CodeT5+, CodeGen2, StarCoder, Code Llama, DeepSeek-Coder, DeepSeek-Coder-V2, and related families [66-79]. Surveys on code LLMs and LLMs for software engineering document the broad task transfer these models support, from generation and translation to summarization, testing, and repair [80-82]. For repair, the key observation is that these models internalize large latent distributions over fixes, making them useful even before task-specific adaptation [72-82].

Prompt-based repair. Zero-shot and few-shot prompting constituted the first LLM-based APR wave. AlphaRepair revisited APR through frozen pretrained models, while Codex- and ChatGPT-based studies showed that large models can often fix benchmark bugs without any explicit repair training [83-88]. Broader evaluations then compared complete-function generation, infilling, line editing, retrieval-augmented prompting, explanation-rich demonstrations, and self-consistent decoding [86-91]. The main finding was not merely that larger models repair more bugs, but that repair performance is highly prompt- and context-sensitive [83-91]. This pushed the literature toward more disciplined in-context design.

Objective alignment and analysis augmentation. Several works argued that code LLMs are hindered when APR is posed as masked infilling or hunk completion that mismatches decoder training objectives. D4C, for example, showed that whole-function debugging with failing-test evidence can outperform more traditional fault-localization-first repair workflows [92]. Similar work enriched prompts with contrastive test pairs, execution traces, static analysis outputs, compiler diagnostics, or runtime feedback, producing models and prompting policies that increasingly resemble automated debugging workflows [93-103, 116, 120-124]. These methods are important because they reposition repair as evidence-conditioned refinement rather than one-shot patching.

Fine-tuning and parameter-efficient adaptation. Parallel work fine-tuned code LLMs directly on bug-fix data or repair-relevant synthetic corpora [61, 101-105]. PEFT techniques such as LoRA and QLoRA made this more tractable [104-105]. Systems including RepairLlama, MOREpair, federated fine-tuning approaches, and repair-specific instruction tuning explored how much task alignment can be achieved with modest adaptation budgets [102-105]. Fine-tuning is often stronger than pure prompting on in-domain bug distributions, but it inherits the classic failure modes of benchmark specialization: training-set bias, localization assumptions, and weak out-of-distribution generalization [61, 101-105, 122].

Reinforcement and process feedback. Since tests, compiler messages, and runtime outcomes provide naturally verifiable signals, repair and debugging have become fertile ground for reinforcement learning

and process supervision. RLTF, StepCoder, process-supervised policy optimization, PRLCoder, and repair systems trained with process-based feedback all use execution-derived signals to densify supervision beyond simple pass/fail rewards [108-115]. These works are motivated by an important limitation of standard LLM post-training: patch correctness is sparse, delayed, and non-differentiable. Process-level rewards approximate the fine-grained debugging judgments humans make when stepping through code. They also foreshadow the shift toward neural debuggers and runtime-grounded world models [50-52, 108-115].

Procedural multi-turn repair. Conversational and test-in-the-loop pipelines occupy the space between static prompts and fully autonomous agents. ChatRepair, ContrastRepair, and related systems alternate generation with test execution, assertion feedback, or structured prompts that help the model refine hypotheses [93-95]. UGen and UTDebug extend this line by generating debugging tests when human-written unit tests are missing or insufficient, then using backtracking and test-time scaling to reduce overfitting to noisy feedback [137]. These approaches are methodologically significant because they show that Code-LLM repair quality depends not only on model knowledge, but also on the design of the outer loop.

Automated debugging with runtime interaction. A separate but converging literature treats debugging itself as the primary task. DebugBench, DebugEval, MLDebugging, MdEval, FeedbackEval, the Precise Debugging Benchmark, MegaBugFix, and related studies evaluate models on localization, bug identification, code review, repair, feedback utilization, multilingual debugging, and edit precision [129-137, 158-165]. Systems such as LDB, MGDebugger, and ChatDBG use runtime execution, trace segmentation, or debugger-mediated interaction to provide stepwise evidence to an LLM [49-52]. These works expose a critical limitation of pure generation-centric repair metrics: high pass rates do not necessarily imply precise debugging. Over-editing and wholesale regeneration can conceal weak localization and poor causal understanding, which is precisely why newer benchmarks measure edit-level precision, bug-level recall, and feedback comprehension rather than only end-state pass/fail outcomes [129-137].

Repository-level and agentic repair. The strongest recent trend is the move from function-level APR toward repository-scale issue resolution. SWE-bench made this transition concrete by requiring systems to resolve real GitHub issues in full repositories [141]. SWE-Agent, Agentless, AutoCodeRover, RepairAgent, OpenHands, MAGIS, SWE-Search, KGCompass, Learn-by-Interact, Marscode Agent, LANTERN, VulDebugger, and related systems treat repair as sequential decision-making over tools, files, tests, and repositories [142-155]. They differ in autonomy and structure. Some rely on scripted retrieval or deterministic loops; others use explicit role decomposition, Monte Carlo tree search, or self-generated trajectories. What unifies them is that they embed the LLM in a software environment, not just a prompt. This is arguably the clearest sign that automated debugging and APR have merged into a common research agenda.

The methodological landscape can therefore be summarized as a progression along two axes. The first axis is **where the repair prior lives**: in templates, in weights, in prompts, or in external repositories and tools. The second axis is **how feedback is integrated**: offline supervision, in-context evidence, procedural loops, or autonomous interaction. Classical APR and modern Code-LLM work are not discontinuous endpoints on these axes; rather, the latter re-instantiate the former at a different scale. Code LLMs supply broader priors and more flexible reasoning, but reliable repair still requires structured evidence, careful control, and principled evaluation [1-42, 83-155].

Challenges and Prospects

The first major challenge is **semantic correctness beyond weak or incomplete tests**. This problem predates LLMs, but model scale intensifies it. Large models are unusually good at generating plausible-looking code that satisfies narrow visible tests while violating deeper invariants, changing unnecessary behavior, or introducing new latent faults [25-26, 127-137]. Recent work on patch correctness assessment, execution-free critics, LLM-as-judge paradigms, and “solved issue” auditing directly addresses this gap, but none fully solves it [125-128]. A plausible prospect is a layered correctness stack combining hidden tests, generated differential tests, static analyzers, type checkers, lightweight symbolic checks, and learned critics. The key research question is not whether any single verifier can certify patches, but how to combine cheap and expensive signals so that verification scales to realistic continuous-integration settings.

The second challenge is **precise debugging versus over-editing**. Code LLMs often regenerate large code regions instead of performing minimal, causally targeted fixes [129-137]. On simple academic benchmarks this may still pass tests, but in real repositories it harms maintainability, reviewability, and trust. The new generation of debugging benchmarks is valuable precisely because it penalizes excessive edits and distinguishes localization from regeneration [129-137]. Future repair systems should optimize not only functional success but also edit discipline: minimality, locality, style preservation, and compatibility with project conventions. This is one area where classical APR’s bias toward small edits may still have something to teach the LLM era [22-28].

The third challenge is **context selection and repository-scale reasoning**. Repository-level issues demand far more than generating a diff from a localized function. They require understanding issue reports, navigating cross-file dependencies, retrieving relevant definitions, and maintaining coherent beliefs across long tool trajectories [117-124, 141-155]. Yet context windows remain finite, retrieval is imperfect, and noise can easily drown the truly relevant evidence. Knowledge graphs, agent memories, structured repository maps, and path-guided retrieval are promising because they organize context before it reaches the model [117, 119, 123-124, 149-150]. A likely prospect is a shift from flat retrieval toward structured evidence compilation: issue-to-code graphs, causal dependency summaries, execution-aware slices, and cached intermediate notes that are optimized for agent use, not just human reading.

The fourth challenge is **robustness, reproducibility, and evaluation validity**. The literature repeatedly shows sensitivity to prompts, sampling temperature, random seed, model version, test budget, and task framing [83-95, 122, 127, 136-137]. Benchmark contamination and hidden overlap with pretraining corpora further complicate interpretation, especially for small debugging datasets or synthetic bug suites [72, 80-82, 141, 166-170]. Newer efforts such as GitBug-Java, BigIssue, MdEval, Precise Debugging Benchmark, and MegaBugFix indicate a healthy methodological turn toward harder, larger, and more realistic evaluation [135-136, 160, 164-165]. Still, the field needs better measurement of cost, latency, edit precision, semantic equivalence, failure diversity, and human review burden. In particular, comparisons across papers remain difficult when they differ in localization assumptions, sampling budgets, visible tests, and repository setup.

The fifth challenge is **economic and systems cost**. Agentic repair can be accurate, but it is expensive in tokens, wall-clock time, and orchestration complexity [141-155]. Multi-turn loops, repeated testing, and tool invocations can make the difference between a promising research prototype and an impractical engineering solution. This cost problem is not peripheral; it shapes what methods can be deployed in CI, IDEs, or education. Research on process supervision, adaptive search depth, cheap critics, staged escalation policies, and fixed-budget retrieval is therefore central rather than auxiliary [108-115, 143-150,

174]. A promising direction is hierarchical repair: attempt lightweight prompted or procedurally guided repair first, escalate to tool-rich agents only when evidence suggests the bug is cross-file, security-critical, or semantically complex.

The sixth challenge is **security, privacy, and trustworthiness**. Vulnerability repair, static-analysis repair, and repository debugging often involve proprietary code, sensitive logs, or security-critical behavior [97-103, 109, 117, 120-128, 153, 170]. This raises concerns about code leakage to external APIs, covert memorization, and silent generation of insecure or policy-violating fixes. Several works on vulnerability repair, static-repair hybrids, federated adaptation, and human-in-the-loop agents point toward safer deployment patterns [97-105, 109, 117, 120-128, 145, 155]. But trustworthy repair likely requires auditable tool traces, policy-gated actions, rollback mechanisms, and explicit uncertainty communication, not just stronger base models.

The seventh challenge is **human integration**. Fully autonomous repair is intellectually compelling, but the most realistic near-term deployments may be mixed-initiative systems that present candidate explanations, localized evidence, or multiple patch options to developers [41, 49, 94, 141-155]. The history of APR already shows that patch acceptability, understandability, and workflow fit matter as much as raw benchmark success [35-42]. Code LLMs offer a unique advantage here: they can explain, compare, and justify. Yet natural-language rationales are themselves unreliable and may diverge from actual patch semantics [49, 125-128]. A productive prospect is therefore **explanation-grounded repair**: explanations that are checked against code diffs, tests, traces, and static facts rather than taken at face value.

Finally, the most interesting scientific prospect may be the emergence of **neural debuggers and world models for code**. Runtime-grounded systems such as LDB, MGDebugger, ChatDBG, UTDebug, and neural debugger prototypes suggest that better repair may come from models that can simulate execution and causal intervention, not just imitate patches [49-52, 137-140]. If code LLMs develop stronger conditional execution models, they may support line-by-line reasoning, counterfactual debugging, reversible state inference, and lower-cost inner-loop evaluation. That would move the field from patch generation with external checking toward something closer to machine-assisted program understanding. Whether such models can scale to repository-level software remains open, but the direction is conceptually important because it reconnects modern Code-LLM research with the long debugging tradition of tracing, stepping, and causal diagnosis [43-52].

Conclusion

Program repair and automated debugging with Code LLMs should not be viewed as a narrow replacement of symbolic or statistical APR by larger neural models. The deeper transformation is methodological. Classical APR taught the field that localization, patch generation, validation, and semantic correctness are distinct problems. Code LLMs add a powerful repair prior, but they do not eliminate those distinctions. Instead, they make it newly possible to integrate them inside unified systems that can reason over text, code, traces, tests, tools, and repositories [1-42, 65-155].

The literature now supports three broad conclusions. First, **feedback is the central organizing principle** of modern repair: tests, traces, compiler messages, static warnings, retrieved history, and debugger state are not optional embellishments but primary signals that shape performance [49-52, 92-128, 137-155]. Second, **outer-loop design matters as much as model design**: procedural and agentic systems outperform naive one-shot repair because they control evidence acquisition and iterative refinement [93-95, 137-155]. Third, **evaluation must move beyond pass/fail patches** toward precise debugging, trustworthy verification, realistic repository settings, and deployability under budget [25-26, 127-137, 141, 160-175].

The next phase of the field will likely be defined by hybridization rather than purity: repository-aware retrieval with precise localization, runtime-grounded debuggers with learned critics, process-supervised training with selective symbolic checks, and mixed-initiative workflows that preserve human oversight while reducing debugging burden. If these lines mature, Code LLMs may become not just patch generators but dependable reasoning components in software maintenance. If they do not, the field risks repeating an older APR pattern: impressive plausible patches, brittle assumptions, and limited real-world adoption. The central research agenda, therefore, is not merely to make models fix more bugs, but to make them debug, justify, and verify repairs in ways that are precise, economical, and trustworthy.

References

- [1] Roderick Bloem, Hadas Chockler, Gregor Grösslinger, and Barbara Jobstmann. “Program Repair as a Game.” In *Computer Aided Verification*, 2005.
- [2] Sebastian Staber, Armin Biere, and Martina Seidl. “Automatic Incremental Repair of a Formula of Computation Tree Logic.” In *International Workshop on Automated Verification of Critical Systems*, 2005.
- [3] Andreas Griesmayer, Roderick Bloem, and Byron Cook. “Repair of Boolean Programs with an Application to C.” In *Computer Aided Verification*, 2006.
- [4] Westley Weimer. “Patches as Better Bug Reports.” In *Generative Programming and Component Engineering*, 2006.
- [5] Gene Novark, Emery D. Berger, and Benjamin G. Zorn. “Exterminator: Automatically Correcting Memory Errors with High Probability.” In *PLDI*, 2007.
- [6] Andrea Arcuri. “On the Automation of Fixing Software Bugs.” In *ICSE Companion*, 2008.
- [7] Andrea Arcuri and Xin Yao. “A Novel Co-evolutionary Approach to Automatic Software Bug Fixing.” In *IEEE Congress on Evolutionary Computation*, 2008.
- [8] Westley Weimer, ThanhVu Nguyen, Claire Le Goues, and Stephanie Forrest. “Automatically Finding Patches Using Genetic Programming.” In *ICSE*, 2009.
- [9] Jeff H. Perkins, Sunghun Kim, Sam Larsen, Saman Amarasinghe, Jonathan Bachrach, Michael Carbin, Carlos Pacheco, Frank Sherwood, Stelios Sidiroglou, Greg Sullivan, Weng-Fai Wong, Yoav Zibin, Michael D. Ernst, and Martin Rinard. “Automatically Patching Errors in Deployed Software.” In *SOSP*, 2009.
- [10] Valentin Dallmeier, Nils Knopp, Christoph Mallon, Sebastian Hack, and Andreas Zeller. “Generating Fixes from Object Behavior Anomalies.” In *ASE*, 2009.
- [11] Brett Daniel, Vilas Jagannath, Danny Dig, and Darko Marinov. “ReAssert: A Tool for Repairing Broken Unit Tests.” In *ASE*, 2009.
- [12] Dennis Jeffrey, Feng Feng, Neelam Gupta, and Rajiv Gupta. “BugFix: A Learning-Based Tool to Assist Developers in Fixing Bugs.” In *ICPC*, 2009.

- [13] Claire Le Goues, ThanhVu Nguyen, Stephanie Forrest, and Westley Weimer. “A Systematic Study of Automated Program Repair: Fixing 55 out of 105 Bugs for \\$8 Each.” In ICSE, 2012.
- [14] Claire Le Goues, ThanhVu Nguyen, Stephanie Forrest, and Westley Weimer. “GenProg: A Generic Method for Automatic Software Repair.” IEEE Transactions on Software Engineering, 38(1):54–72, 2012.
- [15] Dongsun Kim, Jaechang Nam, Jaewoo Song, and Sunghun Kim. “Automatic Patch Generation Learned from Human-Written Patches.” In ICSE, 2013.
- [16] Hoang Duong T. Nguyen, Dawei Qi, Abhik Roychoudhury, and Satish Chandra. “SemFix: Program Repair via Semantic Analysis.” In ICSE, 2013.
- [17] Federico DeMarco, Jigar Mukherjee, and Daniel Grossman. “Automatic Repair of Programs via if Conditions and Missing Preconditions.” In FSE, 2014.
- [18] Earl T. Barr, Yuetang Liu, and Zhendong Su. “The Plastic Surgery Hypothesis.” In FSE, 2014.
- [19] Shalini Kaleeswaran, Varun Tulsian, Aditya Kanade, and Ashish Orso. “MintHint: Automated Synthesis of Repair Hints.” In ICSE, 2014.
- [20] Martin Monperrus. “A Critical Review of ‘Automatic Patch Generation Learned from Human-Written Patches’ : Essay on the Problem Statement and the Evaluation of Automatic Software Repair.” In ICSE, 2014.
- [21] René Just, Darioush Jalali, and Michael D. Ernst. “Defects4J: A Database of Existing Faults to Enable Controlled Testing Studies for Java Programs.” In ISSTA, 2014.
- [22] Fan Long and Martin Rinard. “Staged Program Repair with Condition Synthesis.” In ESEC/FSE, 2015.
- [23] Sergey Mechtaev, Jooyong Yi, and Abhik Roychoudhury. “DirectFix: Looking for Simple Program Repairs.” In ICSE, 2015.
- [24] Shin Hwei Tan and Abhik Roychoudhury. “Relifix: Automated Repair of Software Regressions.” In ICSE, 2015.
- [25] Edward K. Smith, Earl T. Barr, Claire Le Goues, and Yuriy Brun. “Is the Cure Worse than the Disease? Overfitting in Automated Program Repair.” In ESEC/FSE, 2015.
- [26] Zichao Qi, Fan Long, Sara Achour, and Martin Rinard. “An Analysis of Patch Plausibility and Correctness for Generate-and-Validate Patch Generation Systems.” In ISSTA, 2015.
- [27] Stelios Sidiroglou-Douskos, Eric Lahtinen, Fan Long, and Martin Rinard. “Automatic Error Elimination by Horizontal Code Transfer Across Multiple Applications.” In PLDI, 2015.
- [28] Matias Martinez and Martin Monperrus. “Mining Software Repair Models for Reasoning on the Search Space of Automated Program Fixing.” Empirical Software Engineering, 20:1765–1798, 2015.

- [29] Yiming Ke, Kathryn T. Stolee, Claire Le Goues, and Yuriy Brun. “Repairing Programs with Semantic Code Search.” In ASE, 2015.
- [30] Fan Long and Martin Rinard. “Automatic Patch Generation by Learning Correct Code.” In POPL, 2016.
- [31] Sergey Mechtaev, Jooyong Yi, and Abhik Roychoudhury. “Angelix: Scalable Multiline Program Patch Synthesis via Symbolic Analysis.” In ICSE, 2016.
- [32] Xuan Bach D. Le, David Lo, and Claire Le Goues. “History Driven Program Repair.” In SANER, 2016.
- [33] Matias Martinez and Martin Monperrus. “ASTOR: A Program Repair Library for Java.” In ISSTA Demonstrations, 2016.
- [34] Yang Hu, Umair Z. Ahmed, Sergey Mechtaev, Ben Leong, and Abhik Roychoudhury. “Re-Factoring Based Program Repair Applied to Programming Assignments.” In ASE, 2019.
- [35] Claire Le Goues, Michael Pradel, and Abhik Roychoudhury. “Automated Program Repair.” *Communications of the ACM*, 62(12):56–65, 2019.
- [36] Emily Winter, Vesna Nowack, David Bowes, Steve Counsell, Tracy Hall, Sæmundur Haraldsson, and John Woodward. “Let’s Talk with Developers, Not about Developers: A Review of Automatic Program Repair Research.” *IEEE Transactions on Software Engineering*, 49(1):419–436, 2022.
- [37] Kai Huang, Zhengzi Xu, Su Yang, Hongyu Sun, Xuejun Li, Zheng Yan, and Yuqing Zhang. “A Survey on Automated Program Repair Techniques.” *arXiv:2303.18184*, 2023.
- [38] Quanjun Zhang, Chunrong Fang, Yuxiang Ma, Weisong Sun, and Zhenyu Chen. “A Survey of Learning-Based Automated Program Repair.” *ACM Transactions on Software Engineering and Methodology*, 33(2), 2023.
- [39] Kai Huang, Zhengzi Xu, Su Yang, Hongyu Sun, Xuejun Li, Zheng Yan, and Yuqing Zhang. “Evolving Paradigms in Automated Program Repair: Taxonomy, Challenges, and Opportunities.” *ACM Computing Surveys*, 57(2), 2024.
- [40] Sena Dikici and Turgay Tugay Bilgin. “Advancements in Automated Program Repair: A Comprehensive Review.” *Knowledge and Information Systems*, 2025.
- [41] Charaka Geethal, Marcel Böhme, and Van-Thuan Pham. “Human-in-the-Loop Automatic Program Repair.” *IEEE Transactions on Software Engineering*, 49(10):4526–4549, 2023.
- [42] Claire Le Goues, Michael Pradel, Abhik Roychoudhury, and Shin Hwei Tan. “Automated Programming and Program Repair.” *Dagstuhl Reports*, 14(10):39–57, 2025.
- [43] James A. Jones, Mary Jean Harrold, and John Stasko. “Visualization of Test Information to Assist Fault Localization.” In ICSE, 2002.
- [44] Andreas Zeller and Ralf Hildebrandt. “Simplifying and Isolating Failure-Inducing Input.” *IEEE Transactions on Software Engineering*, 28(2):183–200, 2002.

- [45] Holger Cleve and Andreas Zeller. “Locating Causes of Program Failures.” In ICSE, 2005.
- [46] Rui Abreu, Peter Zoetewij, and Arjan J. C. van Gemund. “An Evaluation of Similarity Coefficients for Software Fault Localization.” In PRDC, 2006.
- [47] W. Eric Wong, Vidroha Debroy, and Ruoting Gao. “The DStar Method for Effective Software Fault Localization.” IEEE Transactions on Reliability, 65(1):290–308, 2016.
- [48] Pavel Kassianik, Bogdan Vasilescu, and Prem Devanbu. “BigIssue: A Realistic Bug Localization Benchmark.” arXiv:2207.10739, 2022.
- [49] Kyla Levin, Nicolas van Kempen, Emery D. Berger, and Stephen N. Freund. “ChatDBG: Augmenting Debugging with Large Language Models.” Proceedings of the ACM on Software Engineering, 2(FSE): 1892–1913, 2025. arXiv:2403.16354.
- [50] Li Zhong, Zilong Wang, and Jingbo Shang. “Debug like a Human: A Large Language Model Debugger via Verifying Runtime Execution Step-by-step.” In Findings of ACL, 2024. arXiv:2402.16906.
- [51] Yuling Shi, Songsong Wang, Chengcheng Wan, and Xiaodong Gu. “From Code to Correctness: Closing the Last Mile of Code Generation with Hierarchical Debugging.” In ICSE, 2026. arXiv:2410.01215.
- [52] Maximilian Beck, Jonas Gehring, Jannik Kossen, and Gabriel Synnaeve. “Towards a Neural Debugger for Python.” arXiv:2603.09951, 2026.
- [53] Michele Tufano, Cody Watson, Gabriele Bavota, Massimiliano Di Penta, Martin White, and Denys Poshyvanyk. “An Empirical Study on Learning Bug-Fixing Patches in the Wild via Neural Machine Translation.” ACM Transactions on Software Engineering and Methodology, 28(4), 2019. arXiv: 1812.08693.
- [54] Michele Tufano, Jevgenija Pantiuchina, Cody Watson, Gabriele Bavota, and Denys Poshyvanyk. “On Learning Meaningful Code Changes via Neural Machine Translation.” In ICSE, 2019. arXiv:1901.09102.
- [55] Zimin Chen, Steve Kommrusch, Michele Tufano, Louis-Noël Pouchet, Denys Poshyvanyk, and Martin Monperrus. “SequenceR: Sequence-to-Sequence Learning for End-to-End Program Repair.” IEEE Transactions on Software Engineering, 47(9):1943–1959, 2021. arXiv:1901.01808.
- [56] Thibaud Lutellier, Hung Viet Pham, Lawrence Pang, Yitong Li, Moshi Wei, and Lin Tan. “CoCoNuT: Combining Context-Aware Neural Translation Models Using Ensemble for Program Repair.” In ISSTA, 2020.
- [57] Nan Jiang, Thibaud Lutellier, and Lin Tan. “CURE: Code-Aware Neural Machine Translation for Automatic Program Repair.” In ICSE, 2021. arXiv:2103.00073.
- [58] Qihao Zhu, Zeyu Sun, Yuan-an Xiao, Wenjie Zhang, Kang Yuan, Yingfei Xiong, and Lu Zhang. “A Syntax-Guided Edit Decoder for Neural Program Repair.” In ESEC/FSE, 2021. arXiv:2106.08253.
- [59] He Ye, Matias Martinez, and Martin Monperrus. “Neural Program Repair with Execution-Based Backpropagation.” In ICSE, 2022.

- [60] Nan Jiang, Thibaud Lutellier, Yiling Lou, Lin Tan, Dan Goldwasser, and Xiangyu Zhang. “KNOD: Domain Knowledge Distilled Tree Decoder for Automated Program Repair.” In ICSE, 2023.
- [61] Nan Jiang, Kevin Liu, Thibaud Lutellier, and Lin Tan. “Impact of Code Language Models on Automated Program Repair.” In ICSE, 2023. arXiv:2302.05020.
- [62] Ripon K. Saha, Yingjun Lyu, Hirohisa Aman, Yasutaka Kamei, and Naoyasu Ubayashi. “ELIXIR: Effective Object-Oriented Program Repair.” In ASE, 2017; widely circulated as arXiv:2112.10915.
- [63] Kui Liu, Anil Koyuncu, Dongsun Kim, and Tégawendé F. Bissyandé. “TBar: Revisiting Template-Based Automated Program Repair.” In ISSTA, 2019. arXiv:1903.08409.
- [64] Kui Liu, Anil Koyuncu, Dongsun Kim, and Tégawendé F. Bissyandé. “AVATAR: Fixing Semantic Bugs with Fix Patterns of Static Analysis Violations.” In SANER, 2019. arXiv:1812.07270.
- [65] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. “Attention Is All You Need.” In NeurIPS, 2017.
- [66] Wasi Ahmad, Saikat Chakraborty, Baishakhi Ray, and Kai-Wei Chang. “Unified Pre-training for Program Understanding and Generation.” In NAACL-HLT, 2021.
- [67] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, et al. “CodeBERT: A Pre-Trained Model for Programming and Natural Languages.” In Findings of EMNLP, 2020.
- [68] Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie Liu, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, Michele Tufano, et al. “GraphCodeBERT: Pre-Training Code Representations with Data Flow.” arXiv:2009.08366, 2021.
- [69] Yue Wang, Weishi Wang, Shafiq Joty, and Steven C. H. Hoi. “CodeT5: Identifier-Aware Unified Pre-trained Encoder-Decoder Models for Code Understanding and Generation.” arXiv:2109.00859, 2021.
- [70] Daya Guo, Shuai Lu, Nan Duan, Yanlin Wang, Ming Zhou, and Jian Yin. “UniXcoder: Unified Cross-Modal Pre-training for Code Representation.” In ACL, 2022.
- [71] Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida Wang, Eric Wallace, Freda Shi, Ruiqi Zhong, Scott Yih, Luke Zettlemoyer, and Mike Lewis. “InCoder: A Generative Model for Code Infilling and Synthesis.” In ICLR, 2022.
- [72] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. “Evaluating Large Language Models Trained on Code.” arXiv:2107.03374, 2021.
- [73] Yue Wang, Hung Le, Akhilesh Deepak Gotmare, Nghi DQ Bui, Junnan Li, and Steven C. H. Hoi. “CodeT5+: Open Code Large Language Models for Code Understanding and Generation.” arXiv:2305.07922, 2023.
- [74] Erik Nijkamp, Hiroaki Hayashi, Caiming Xiong, Silvio Savarese, and Yingbo Zhou. “CodeGen2: Lessons for Training LLMs on Programming and Natural Languages.” arXiv:2305.02309, 2023.

- [75] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, et al. “StarCoder: May the Source Be with You!” arXiv:2305.06161, 2023.
- [76] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, et al. “Code Llama: Open Foundation Models for Code.” arXiv:2308.12950, 2023.
- [77] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. “The Llama 3 Herd of Models.” arXiv:2407.21783, 2024.
- [78] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Yu Wu, Y. K. Li, et al. “DeepSeek-Coder: When the Large Language Model Meets Programming—The Rise of Code Intelligence.” arXiv:2401.14196, 2024.
- [79] Qihao Zhu, Daya Guo, Zhihong Shao, Dejian Yang, Peiyi Wang, Runxin Xu, Y. Wu, Yukun Li, Huazuo Gao, Shirong Ma, et al. “DeepSeek-Coder-V2: Breaking the Barrier of Closed-Source Models in Code Intelligence.” arXiv:2406.11931, 2024.
- [80] Juyong Jiang, Fan Wang, Jiasi Shen, Sungju Kim, and Sunghun Kim. “A Survey on Large Language Models for Code Generation.” arXiv:2406.00515, 2024.
- [81] Angela Fan, Beliz Gokkaya, Mark Harman, Mitya Lyubarskiy, Shubho Sengupta, Shin Yoo, and Jie M. Zhang. “Large Language Models for Software Engineering: Survey and Open Problems.” In ICSE-FoSE, 2023.
- [82] Junjie Wang, Yuchao Huang, Chunyang Chen, Zhe Liu, Song Wang, and Qing Wang. “Software Testing with Large Language Models: Survey, Landscape, and Vision.” IEEE Transactions on Software Engineering, 50(4):911–936, 2024. arXiv:2307.07221.
- [83] Chunqiu Steven Xia and Lingming Zhang. “Less Training, More Repairing Please: Revisiting Automated Program Repair via Zero-Shot Learning.” In ESEC/FSE, 2022.
- [84] Julian Aron Prenner, Hlib Babii, and Romain Robbes. “Can OpenAI’s Codex Fix Bugs? An Evaluation on QuixBugs.” In Workshop on Automated Program Repair, 2022.
- [85] Zhiyu Fan, Xiang Gao, Martin Mirchev, Abhik Roychoudhury, and Shin Hwei Tan. “Automated Repair of Programs from Large Language Models.” In ICSE, 2023.
- [86] Chunqiu Steven Xia, Yuxiang Wei, and Lingming Zhang. “Automated Program Repair in the Era of Large Pre-trained Language Models.” In ICSE, 2023.
- [87] Haoye Tian, Weiqi Lu, Tsz On Li, Xunzhu Tang, Shing-Chi Cheung, Jacques Klein, and Tégawendé F. Bissyandé. “Is ChatGPT the Ultimate Programming Assistant—How Far Is It?” arXiv:2304.11938, 2023.
- [88] Dominik Sobania, Martin Briesch, Carol Hanna, and Justyna Petke. “An Analysis of the Automatic Bug Fixing Performance of ChatGPT.” In Workshop on Automated Program Repair, 2023.

- [89] Noor Nashid, Mifta Sintaha, and Ali Mesbah. “Retrieval-Based Prompt Selection for Code-Related Few-Shot Learning.” In ICSE, 2023.
- [90] Toufique Ahmed and Premkumar Devanbu. “Majority Rule: Better Patching via Self-Consistency.” arXiv:2306.00108, 2023.
- [91] Shuzheng Gao, Xin-Cheng Wen, Cuiyun Gao, Wenxuan Wang, Hongyu Zhang, and Michael R. Lyu. “What Makes Good In-Context Demonstrations for Code Intelligence Tasks with LLMs?” In ASE, 2023.
- [92] Junjielong Xu, Ying Fu, Shin Hwei Tan, and Pinjia He. “Aligning the Objective of LLM-based Program Repair.” arXiv:2404.08877, 2024; later in ICSE, 2025.
- [93] Jiaolong Kong, Mingfei Cheng, Xiaofei Xie, Shangqing Liu, Xiaoning Du, and Qi Guo. “ContrastRepair: Enhancing Conversation-Based Automated Program Repair via Contrastive Test Case Pairs.” arXiv: 2403.01971, 2024.
- [94] Chunqiu Steven Xia and Lingming Zhang. “Automated Program Repair via Conversation: Fixing 162 out of 337 Bugs for \\$0.42 Each Using ChatGPT.” In ISSTA, 2024.
- [95] Islem Bouzenia, Yangruibo Ding, Kexin Pei, Baishakhi Ray, and Michael Pradel. “TraceFixer: Execution Trace-Driven Program Repair.” arXiv:2304.12743, 2023.
- [96] Matthew Jin, Syed Shahriar, Michele Tufano, Xin Shi, Shuai Lu, Neel Sundaresan, and Alexey Svyatkovskiy. “InferFix: End-to-End Program Repair with LLMs.” In ESEC/FSE, 2023.
- [97] Yiu Wai Chow, Luca Di Grazia, and Michael Pradel. “PyTy: Repairing Static Type Errors in Python.” In ICSE, 2024.
- [98] Pei Liu, Bo Lin, Yihao Qin, Cheng Weng, and Liqian Chen. “T-RAP: A Template-Guided Retrieval-Augmented Vulnerability Patch Generation Approach.” In Internetware, 2024.
- [99] Xin Zhou, Kisub Kim, Bowen Xu, DongGyun Han, and David Lo. “Out of Sight, Out of Mind: Better Automatic Vulnerability Repair by Broadening Input Ranges and Sources.” In ICSE, 2024.
- [100] Michael Fu, Chakkrit Tantithamthavorn, Trung Le, Van Nguyen, and Dinh Phung. “VulRepair: A T5-Based Automated Software Vulnerability Repair.” In ESEC/FSE, 2022.
- [101] Fengjie Li, Jiajun Jiang, Jiajun Sun, and Hongyu Zhang. “Hybrid Automated Program Repair by Combining Large Language Models and Program Analysis.” ACM Transactions on Software Engineering and Methodology, 2024.
- [102] Boyang Yang, Haoye Tian, Jiadong Ren, Hongyu Zhang, Jacques Klein, Tégawendé Bissyandé, Claire Le Goues, and Shunfu Jin. “MOREpair: Teaching LLMs to Repair Code via Multi-Objective Fine-Tuning.” ACM Transactions on Software Engineering and Methodology, 2025.
- [103] André Silva, Sen Fang, and Martin Monperrus. “RepairLlama: Efficient Representations and Fine-Tuned Adapters for Program Repair.” arXiv:2312.15698, 2023.
- [104] Guochang Li, Chen Zhi, Jialiang Chen, Junxiao Han, and Shuiguang Deng. “Exploring Parameter-Efficient Fine-Tuning of Large Language Model on Automated Program Repair.” In ASE, 2024.

- [105] Wenqiang Luo, Jacky Keung, Boyang Yang, He Ye, Claire Le Goues, Tégawendé Bisseyandé, Haoye Tian, and Xuan Bach D. Le. “When Fine-Tuning LLMs Meets Data Privacy: An Empirical Study of Federated Learning in LLM-Based Program Repair.” *ACM Transactions on Software Engineering and Methodology*, 2024.
- [106] Nan Jiang and Yi Wu. “RepairCAT: Applying Large Language Model to Fix Bugs in AI-Generated Programs.” In *Workshop on Automated Program Repair*, 2024.
- [107] Fernando Vallecillos Ruiz, Max Hort, and Leon Moonen. “The Art of Repair: Optimizing Iterative Program Repair with Instruction-Tuned Models.” *arXiv:2505.02931*, 2025.
- [108] Yuze Zhao, Zhenya Huang, Yixiao Ma, Rui Li, Kai Zhang, Hao Jiang, Qi Liu, Linbo Zhu, and Yu Su. “Repair: Automated Program Repair with Process-based Feedback.” *arXiv:2408.11296*, 2024.
- [109] Nafis Tanveer Islam, Joseph Khoury, Andrew Seong, Mohammad Bahrami Karkevandi, Gonzalo de la Torre Parra, Elias Bou-Harb, and Peyman Najafirad. “LLM-Powered Code Vulnerability Repair with Reinforcement Learning and Semantic Reward.” *arXiv:2401.03374*, 2024.
- [110] Yuxiang Wei, Olivier Duchenne, Jade Copet, Quentin Carbonneaux, Lingming Zhang, Daniel Fried, Gabriel Synnaeve, Rishabh Singh, and Sida I. Wang. “SWE-RL: Advancing LLM Reasoning via Reinforcement Learning on Open Software Evolution.” *arXiv:2502.18449*, 2025.
- [111] Zhenlong Dai, Bingrui Chen, Zhuoluo Zhao, Xiu Tang, Sai Wu, Chang Yao, Zhipeng Gao, and Jingyuan Chen. “Less Is More: Adaptive Program Repair with Bug Localization and Preference Learning.” In *AAAI*, 2025.
- [112] Shihan Dou, Yan Liu, Haoxiang Jia, Enyu Zhou, Limao Xiong, Junjie Shan, Caishuang Huang, Xiao Wang, Xiaoran Fan, Zhiheng Xi, et al. “StepCoder: Improving Code Generation with Reinforcement Learning from Compiler Feedback.” In *ACL*, 2024.
- [113] Jiate Liu, Yiqin Zhu, Kaiwen Xiao, Qiang Fu, Xiao Han, Wei Yang, and Deheng Ye. “RLTF: Reinforcement Learning from Unit Test Feedback.” *arXiv:2307.04349*, 2023.
- [114] Ning Dai, Zheng Wu, Renjie Zheng, Ziyun Wei, Wenlei Shi, Xing Jin, Guanlin Liu, Chen Dun, Liang Huang, and Lin Yan. “Process Supervision-Guided Policy Optimization for Code Generation.” *arXiv:2410.17621*, 2024.
- [115] Yufan Ye, Ting Zhang, Wenbin Jiang, and Hua Huang. “Process-Supervised Reinforcement Learning for Code Generation.” *arXiv:2502.01715*, 2025.
- [116] Yuxiang Wei, Chunqiu Steven Xia, and Lingming Zhang. “Copiloting the Copilots: Fusing Large Language Models with Completion Engines for Automated Program Repair.” In *ESEC/FSE*, 2023. *arXiv:2309.00608*.
- [117] Yuxiao Chen, Jingzheng Wu, Xiang Ling, Changjiang Li, Zhiqing Rui, Tianyue Luo, and Yanjun Wu. “When Large Language Models Confront Repository-Level Automatic Program Repair: How Well They Done?” In *ICSE Companion*, 2024.
- [118] Elijah Mansur, Johnson Chen, Muhammad Anas Raza, and Mohammad Wardat. “RAGFix: Enhancing LLM Code Repair Using RAG and Stack Overflow Posts.” In *IEEE BigData*, 2024.

- [119] Shuyin Ouyang, Jie Zhang, Zeyu Sun, and Albert Merono Penuela. “Knowledge-Enhanced Program Repair for Data Science Code.” In ICSE, 2025.
- [120] Mirazul Haque, Petr Babkin, Farima Farmahinifarahani, and Manuela Veloso. “Towards Effectively Leveraging Execution Traces for Program Repair with Code LLMs.” In Knowledge-Augmented Methods for NLP Workshop, 2025.
- [121] Greta Dolcetti, Vincenzo Arceri, Eleonora Iotti, Sergio Maffei, Agostino Cortesi, and Enea Zaffanella. “Helping LLMs Improve Code Generation Using Feedback from Testing and Static Analysis.” arXiv:2412.14841, 2025.
- [122] Fengjie Li, Jiajun Jiang, Jiajun Sun, and Hongyu Zhang. “Evaluating the Generalizability of LLMs in Automated Program Repair.” arXiv:2503.09217, 2025.
- [123] Nikhil Parasaram, Huijie Yan, Boyu Yang, Zineb Flahy, Abriele Qudsi, Damian Ziaber, Earl Barr, and Sergey Mechtaev. “The Fact Selection Problem in LLM-Based Program Repair.” arXiv:2404.05520, 2024.
- [124] Tyler Holloway, Ethan R. Elenberg, and AI Permanence. “On the Role of Context Granularity in LLM-Driven Program Repair.” 2024.
- [125] Aashish Yadavally, Hoan Nguyen, Laurent Callot, and Gauthier Guinet. “Large Language Model Critics for Execution-Free Evaluation of Code Changes.” arXiv:2501.16655, 2025.
- [126] Xin Zhou, Bowen Xu, Kisub Kim, DongGyun Han, Hung Huu Nguyen, Thanh Le-Cong, Junda He, Bach Le, and David Lo. “Leveraging Large Language Model for Automatic Patch Correctness Assessment.” IEEE Transactions on Software Engineering, 2024.
- [127] You Wang, Michael Pradel, and Zhongxin Liu. “Are ‘Solved Issues’ in SWE-bench Really Solved Correctly? An Empirical Study.” arXiv:2503.15223, 2025.
- [128] Junda He, Jieke Shi, Terry Yue Zhuo, Christoph Treude, Jiamou Sun, Zhenchang Xing, Xiaoning Du, and David Lo. “From Code to Courtroom: LLMs as the New Software Judges.” arXiv:2503.02246, 2025.
- [129] Federico Cassano, Luisa Li, Akul Sethi, Noah Shinn, Abby Brennan-Jones, Jacob Ginesin, Edward Berman, George Chakraborty, Anton Lozhkov, Carolyn Jane Anderson, and Arjun Guha. “Can It Edit? Evaluating the Ability of Large Language Models to Follow Code Editing Instructions.” arXiv:2312.12450, 2023.
- [130] Jiawei Guo, Ziming Li, Xueling Liu, Kaijing Ma, Tianyu Zheng, Zhouliang Yu, Ding Pan, Yizhi Li, Ruibo Liu, Yue Wang, Shuyue Guo, Xingwei Qu, Xiang Yue, Ge Zhang, Wenhui Chen, and Jie Fu. “CodeEditorBench: Evaluating Code Editing Capability of Large Language Models.” arXiv:2404.03543, 2024.
- [131] Runxin Tian, Yifei Wang, et al. “Evaluating Debugging Capability of Large Language Models.” In Findings of ACL, 2024.
- [132] Weinan Yang, et al. “COAST: Enhancing the Code Debugging Ability of LLMs through Multi-Agent Collaboration.” In Findings of NAACL, 2025. DebugEval benchmark introduced therein. arXiv:2408.05006.

- [133] Jinyang Huang, Xiachong Feng, Qiguang Chen, Hanjie Zhao, Zihui Cheng, Jiesong Bai, Jingxuan Zhou, Min Li, and Libo Qin. “MLDebugging: Towards Benchmarking Code Debugging Across Multi-Library Scenarios.” arXiv:2506.13824, 2025.
- [134] Dekun Dai, MingWei Liu, Anji Li, Jialun Cao, Yanlin Wang, Chong Wang, Xin Peng, and Zibin Zheng. “FeedbackEval: A Benchmark for Evaluating Large Language Models in Feedback-Driven Code Repair Tasks.” arXiv:2504.06939, 2025.
- [135] Wang Bill Zhu, Miaosen Chai, Shangshang Wang, YeJia Liu, Song Bian, Honghua Dong, Willie Neiswanger, and Robin Jia. “Precise Debugging Benchmark: Is Your Model Debugging or Regenerating?” arXiv:2604.17338, 2026.
- [136] Balázs Szalontai, Ábel Szauter, Balázs Márton, Péter Verebics, Balázs Pintér, and Tibor Gregorics. “Diff-Based Code Corruption Using LLMs for Large-Scale Bugfix Benchmarking.” arXiv:2606.29088, 2026.
- [137] Muntasir Adnan and Carlos C. N. Kuhn. “The Debugging Decay Index: Rethinking Debugging Strategies for Code LLMs.” arXiv:2506.18403, 2025.
- [138] Archiki Prasad, Elias Stengel-Eskin, Justin Chih-Yao Chen, Zaid Khan, and Mohit Bansal. “Learning to Generate Unit Tests for Automated Debugging.” arXiv:2502.01619, 2025.
- [139] Chao Lei, Yanchuan Chang, Nir Lipovetzky, and Krista A. Ehinger. “A Large Language Model Programming Workflow.” arXiv:2411.14503, 2024.
- [140] Kaijing Li et al. “Instruction Tuning Large Language Models for Code Editing.” In ACL Student Research Workshop, 2024.
- [141] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?” In ICLR, 2024.
- [142] John Yang, Carlos Jimenez, Kilian Lieret, et al. “SWE-Agent: Agent-Computer Interfaces Enable Automated Software Engineering.” In NeurIPS, 2024.
- [143] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. “Agentless: Demystifying LLM-Based Software Engineering Agents.” arXiv:2407.01489, 2024.
- [144] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. “AutoCodeRover: Autonomous Program Improvement.” In ISSTA, 2024.
- [145] Islem Bouzenia, Premkumar Devanbu, and Michael Pradel. “RepairAgent: An Autonomous, LLM-Based Agent for Program Repair.” arXiv:2403.17134, 2024.
- [146] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, et al. “OpenHands: An Open Platform for AI Software Developers as Generalist Agents.” In ICLR, 2025.
- [147] Wei Tao, Yucheng Zhou, Yanlin Wang, Wenqiang Zhang, Hongyu Zhang, and Yu Cheng. “MAGIS: LLM-Based Multi-Agent Framework for GitHub Issue Resolution.” In NeurIPS, 2024.

- [148] Antonis Antoniadis, Albert Örwall, Kexun Zhang, Yuxi Xie, Anirudh Goyal, and William Wang. “SWE-Search: Enhancing Software Agents with Monte Carlo Tree Search and Iterative Refinement.” arXiv:2410.20285, 2024.
- [149] Hongjin Su, Ruoxi Sun, Jinsung Yoon, Pengcheng Yin, Tao Yu, and Serkan Ö. Arık. “Learn-by-Interact: A Data-Centric Framework for Self-Adaptive Agents in Realistic Environments.” arXiv:2501.10893, 2025.
- [150] Boyang Yang, Haoye Tian, Jiadong Ren, Shunfu Jin, Yang Liu, Feng Liu, and Bach Le. “Enhancing Repository-Level Software Repair via Repository-Aware Knowledge Graphs.” arXiv:2503.21710, 2025.
- [151] Yizhou Liu, Pengfei Gao, Xinchun Wang, Jie Liu, Yexuan Shi, Zhao Zhang, and Chao Peng. “Marscode Agent: AI-Native Automated Bug Fixing.” arXiv:2409.00899, 2024.
- [152] Wenqiang Luo, Jacky Wai Keung, Boyang Yang, Jacques Klein, Tégawendé F. Bissyandé, Haoye Tian, and Bach Le. “Unlocking LLM Repair Capabilities in Low-Resource Programming Languages Through Cross-Language Translation and Multi-Agent Refinement.” arXiv:2503.22512, 2025.
- [153] Zhengyao Liu, Yunlong Ma, Jingxuan Xu, Junchen Ai, Xiang Gao, Hailong Sun, and Abhik Roychoudhury. “Agent That Debugs: Dynamic State-Guided Vulnerability Repair.” arXiv:2504.07634, 2025.
- [154] John Yang, Carlos E. Jimenez, Alex L. Zhang, Kilian Lieret, Joyce Yang, Xindi Wu, Ori Press, Niklas Muennighoff, Gabriel Synnaeve, Karthik R. Narasimhan, et al. “SWE-bench Multimodal: Do AI Systems Generalize to Visual Software Domains?” arXiv:2410.03859, 2024.
- [155] Wannita Takerngsaksiri, Jirat Pasuksmit, Patanamon Thongtanunam, Chakkrit Tantithamthavorn, Ruixiong Zhang, Fan Jiang, Jing Li, Evan Cook, Kun Chen, and Ming Wu. “Human-In-the-Loop Software Development Agents.” arXiv:2411.12924, 2024.
- [156] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. “Program Synthesis with Large Language Models.” arXiv:2108.07732, 2021.
- [157] Niklas Muennighoff, Qian Liu, Armel Zebaze, Qinkai Zheng, Binyuan Hui, Terry Yue Zhuo, Swayam Singh, Xiangru Tang, Leandro von Werra, and Shayne Longpre. “OctoPack: Instruction Tuning Code Large Language Models.” In NeurIPS Workshop on Instruction Tuning and Instruction Following, 2023.
- [158] Yuhang Lai, Chengxi Li, Yiming Wang, Tianyi Zhang, Ruiqi Zhong, Luke Zettlemoyer, Wen-tau Yih, Daniel Fried, Sida Wang, and Tao Yu. “DS-1000: A Natural and Reliable Benchmark for Data Science Code Generation.” In ICML, 2023.
- [159] Mohammad Abdullah Matin Khan, M. Saiful Bari, Do Long, Weishi Wang, Md Rizwan Parvez, and Shafiq Joty. “xCodeEval: An Execution-Based Large Scale Multilingual Multitask Benchmark for Code Understanding, Generation, Translation and Retrieval.” In ACL, 2024.
- [160] André Silva, Nuno Saavedra, and Martin Monperrus. “GitBug-Java: A Reproducible Benchmark of Recent Java Bugs.” In MSR, 2024.

- [161] Fernanda Madeiral, Simon Urli, Marcelo Maia, and Martin Monperrus. “BEARS: An Extensible Java Bug Benchmark for Automatic Program Repair Studies.” In SANER, 2019.
- [162] Claire Le Goues, Neal Holtschulte, Edward K. Smith, Yuriy Brun, Premkumar Devanbu, Stephanie Forrest, and Westley Weimer. “The ManyBugs and IntroClass Benchmarks for Automated Repair of C Programs.” IEEE Transactions on Software Engineering, 41(12):1236–1256, 2015.
- [163] Derrick Lin, James Koppel, Angela Chen, and Armando Solar-Lezama. “QuixBugs: A Multi-Lingual Program Repair Benchmark Set Based on the Quixey Challenge.” In SPLASH Companion, 2017.
- [164] Julian Aron Prenner and Romain Robbes. “RunBugRun—An Executable Dataset for Automated Program Repair.” arXiv:2304.01102, 2023.
- [165] Yizhou Liu et al. “MdEval: Massively Multilingual Code Debugging.” arXiv:2411.02310, 2024.
- [166] Juntong Wang et al. “A Survey of Benchmarks for Code Large Language Models and Agents.” arXiv:2505.05283, 2025.
- [167] Quanjun Zhang, Chunrong Fang, Yang Xie, Yuxiang Ma, Weisong Sun, Yun Yang, and Zhenyu Chen. “A Systematic Literature Review on Large Language Models for Automated Program Repair.” arXiv: 2405.01466, 2024.
- [168] Fida Zubair, Maryam Al-Hitmi, and Cagatay Catal. “The Use of Large Language Models for Program Repair.” Computer Standards & Interfaces, 2024.
- [169] Avinash Anand, Akshit Gupta, Nishchay Yadav, and Shaurya Bajaj. “A Comprehensive Survey of AI-Driven Advancements and Techniques in Automated Program Repair and Code Generation.” arXiv: 2411.07586, 2024.
- [170] Xin Zhou, Sicong Cao, Xiaobing Sun, and David Lo. “Large Language Model for Vulnerability Detection and Repair: Literature Review and the Road Ahead.” ACM Transactions on Software Engineering and Methodology, 2024.
- [171] Juyong Jiang, Fan Wang, Jiasi Shen, Sungju Kim, and Sunghun Kim. “A Survey on Large Language Models for Code Generation.” arXiv:2406.00515, 2024.
- [172] Junjie Wang, Yuchao Huang, Chunyang Chen, Zhe Liu, Song Wang, and Qing Wang. “Software Testing with Large Language Models: Survey, Landscape, and Vision.” IEEE Transactions on Software Engineering, 50(4):911–936, 2024.
- [173] Viviana Terragni, Annie Vella, et al. “The Future of AI-Driven Software Engineering.” ACM Transactions on Software Engineering and Methodology, 34(5), 2025.
- [174] Hao Tang, Keya Hu, Jin Zhou, Si Cheng Zhong, Wei-Long Zheng, Xujie Si, and Kevin Ellis. “Code Repair with LLMs Gives an Exploration-Exploitation Tradeoff.” In NeurIPS, 2024.
- [175] Michael Pradel, et al. “Towards Practical and Useful Automated Program Repair for Developers.” 2024.
-